statements based on the input, the resulting SQL statement performs actions other than those the application intended. SQL Injection results from failure of the application to appropriately validate input.

Extended Description

When specially crafted user-controlled input consisting of SQL syntax is used without proper validation as part of SQL queries, it is possible to glean information from the database in ways not envisaged during application design. Depending upon the database and the design of the application, it may also be possible to leverage injection to have the database execute system-related commands of the attackers' choice. SQL Injection enables an attacker to interact directly to the database, thus bypassing the application completely. Successful injection can cause information disclosure as well as ability to add or modify data in the database.

Likelihood Of Attack

High

Typical Severity

High

Relationships

Nature	Type	ID	Name
ChildOf	M	248	Command Injection
ParentOf	D	7	Blind SQL Injection
ParentOf	D	108	Command Line Execution through SQL Injection
ParentOf	D	109	Object Relational Mapping Injection
ParentOf	D	110	SQL Injection through SOAP Parameter Tampering
ParentOf	D	470	Expanding Control over the Operating System from the Database

View Name	Top Level Categories
Domains of Attack	Software
Mechanisms of Attack	Inject Unexpected Items

Execution Flow

Explore

Survey application: The attacker first takes an inventory of the functionality exposed by the application.

Techniques
Spider web sites for all available links
Sniff network communications with application using a utility such as WireShark.

Experiment

WebScarab,etc. to modify HTTP POST parameters, hidden fields, non-freeform fields, etc.
Use network-level packet injection tools such as netcat to inject input
Use modified client (modified by reverse engineering) to inject input.

2. **Experiment with SQL Injection vulnerabilities:** After determining that a given input is vulnerable to SQL Injection, hypothesize what the underlying query looks like. Iteratively try to add logic to the query to extract information from the database, or to modify or delete information in the database.

Techniques
Use public resources such as "SQL Injection Cheat Sheet" at http://ferruh.mavituna.com/makale/sql-injection-cheatsheet/, and try different approaches for adding logic to SQL queries.
Add logic to query, and use detailed error messages from the server to debug the query. For example, if adding a single quote to a query causes an error message, try : "' OR 1=1; --", or something else that would syntactically complete a hypothesized query. Iteratively refine the query.
Use "Blind SQL Injection" techniques to extract information about the database schema.
If a denial of service attack is the goal, try stacking queries. This does not work on all platforms (most notably, it does not work on Oracle or MySQL). Examples of inputs to try include: "'; DROP TABLE SYSOBJECTS; --" and "'); DROP TABLE SYSOBJECTS; --". These particular queries will likely not work because the SYSOBJECTS table is generally protected.

Exploit

Exploit SQL Injection vulnerability: After refining and adding various logic to SQL queries, craft and execute the underlying SQL query that will be used to attack the target system. The goal is to reveal, modify, and/or delete database data, using the knowledge obtained in the previous step. This could entail crafting and executing multiple SQL queries if a denial of service attack is the intent.

Techniques
Craft and Execute underlying SQL query

Prerequisites

SQL queries used by the application to store, retrieve or modify data.

User-controllable input that is not properly validated by the application as part of SQL queries.

Too many false or invalid queries to the database, especially those caused by malformed input.

Consequences

Scope	Impact	Likelihood
Integrity	Modify Data	
Confidentiality	Read Data	
Confidentiality Integrity Availability	Execute Unauthorized Commands	
Confidentiality Access Control Authorization	Gain Privileges	

Mitigations

Strong input validation - All user-controllable input must be validated and filtered for illegal characters as well as SQL content. Keywords such as UNION, SELECT or INSERT must be filtered in addition to characters such as a single-quote(') or SQL-comments (--) based on the context in which they appear.

Use of parameterized queries or stored procedures - Parameterization causes the input to be restricted to certain domains, such as strings or integers, and any input outside such domains is considered invalid and the query fails. Note that SQL Injection is possible even in the presence of stored procedures if the eventual query is constructed dynamically.

Use of custom error pages - Attackers can glean information about the nature of queries from descriptive error messages. Input validation must be coupled with customized error pages that inform about an error without disclosing information about the database or application.

Example Instances

With PHP-Nuke versions 7.9 and earlier, an attacker can successfully access and modify data, including sensitive contents such as usernames and password hashes, and compromise the application through SQL Injection. The protection mechanism against SQL Injection employs a denylist approach to input validation. However, because of an improper denylist, it is possible to inject content such as "foo'/**/UNION" or "foo UNION/**/" to bypass validation and glean sensitive information from the database. See also: CVE-2006-5525

Related Weaknesses

Relevant to the OWASP taxonomy mapping

Entry Name
SQL Injection

References

[REF-607] "OWASP Web Security Testing Guide". Testing for SQL Injection. The Open Web Application Security Project (OWASP). <https://owasp.org/www-project-web-security-testing-guide/latest/4-Web_Application_Security_Testing/07-Input_Validation_Testing/05-Testing_for_SQL_Injection.html>.

Content History

Page Last Updated or Reviewed: July 31, 2018